

Machine Learning с нуля

Неделя 4 - preprocessing, feature engineering и Pipeline

Формат

1-2 часа в день (можно больше).

Каждый день: понять -> изучить -> сделать -> проверить
-> закоммитить.

Инструменты

Python 3.12+ • Jupyter/Colab • pandas • matplotlib •
scikit-learn • Git • GitHub.

Зачем нужна четвёртая неделя?

На третьей неделе модель получала уже готовые числовые данные. В реальных таблицах есть пропуски, текстовые категории и признаки разных масштабов. Если обработать их до разделения данных или обучить преобразование на test, оценка станет слишком оптимистичной.

Главное правило недели: preprocessing обучается только на train

- Сначала отдели test, затем все шаги с fit выполняй только внутри train.
- Объединяй преобразования и модель в Pipeline, чтобы один и тот же процесс работал в CV и на новых данных.
- Сравнивай решения на одинаковых разбиениях и одинаковых метриках.
- Финальный test используй один раз - после выбора признаков и параметров.

Минимум ML для этой недели

- различать числовые и категориальные признаки;
- заполнять пропуски через `SimpleImputer`;
- масштабировать числовые признаки через `StandardScaler`;
- кодировать категории через `OneHotEncoder(handle_unknown='ignore')`;
- применять разные преобразования колонок через `ColumnTransformer`;
- собирать preprocessing и модель в единый `Pipeline`;
- создавать признаки без target leakage;
- оценивать модель через `StratifiedKFold` и несколько метрик;
- подбирать параметры через `GridSearchCV` только на train;
- фиксировать случайность и получать воспроизводимый результат.

Итог за неделю: папка week-4 с семью ноутбуками, переиспользуемой функцией feature engineering, графиками, сравнением baseline и pipeline, результатами CV и коротким отчётом.

Что должно лежать на GitHub к концу недели

Файл/папка	Зачем нужен
README.md	цель, запуск, ссылки на week-4 и итоговые метрики
requirements.txt	воспроизводимые версии библиотек
week-4/notebooks/	7 ежедневных ноутбуков, которые идут сверху вниз
week-4/src/features.py	чистые функции для новых признаков
week-4/figures/	графики пропусков, CV и финальной оценки
week-4/reports/week04_summary.md	короткий вывод по данным, pipeline, CV и test

Правило сдачи: ссылка ведёт на pull request или ветку week-4. По диффу должно быть видно только работу четвёртой недели и необходимые обновления документации.

Карта недели

День	Тема	Результат дня	Что на Git
1	Аудит данных и протокол оценки	Определены target, типы колонок, split и baseline	day01_preprocessing_audit.ipynb
2	Пропуски и масштабирование	Числовые признаки обрабатываются без leakage	day02_numeric_preprocessing.ipynb
3	Категориальные признаки	Категории кодируются и неизвестные значения не ломают predict	day03_categorical_encoding.ipynb
4	ColumnTransformer и Pipeline	Собран единый воспроизводимый pipeline	day04_columntransformer_pipeline.ipynb
5	Feature engineering	Добавлены осмысленные признаки, есть честное сравнение	day05_feature_engineering.ipynb + src/features.py
6	CV, метрики и tuning	Параметры выбраны только по train CV	day06_cv_tuning.ipynb + CV-график
7	Финальный мини-проект	Один test, отчёт и готовый PR	day07_final_project.ipynb + week04_summary.md

Датасет недели

Основной пример - Titanic из `data/titanic.csv`. Target: Survived. В данных есть числовые признаки, категории и пропуски, поэтому на одном датасете можно пройти весь preprocessing-процесс.

- Не использовать PassengerId как полезный сигнал: это идентификатор, а не характеристика пассажира.
- Не передавать Survived в функции, создающие признаки.
- До финального дня сравнивать варианты только на train через одинаковую CV.
- Если файла нет, зафиксировать официальный или стабильный источник и инструкцию загрузки в README.

Дальше - подробности по каждому дню: что понять, что сделать руками, как проверить результат и что обязательно добавить в Git.

Структура репозитория

Продолжай работать в том же репозитории. Для новой недели создай отдельную папку week-4.

```
machine-learning/  
README.md  
LEARNING_LOG.md  
requirements.txt  
.gitignore  
data/titanic.csv  
week-4/  
notebooks/  
day01_preprocessing_audit.ipynb  
day02_numeric_preprocessing.ipynb  
day03_categorical_encoding.ipynb  
day04_columntransformer_pipeline.ipynb  
day05_feature_engineering.ipynb  
day06_cv_tuning.ipynb  
day07_final_project.ipynb  
src/features.py  
figures/  
reports/week04_summary.md
```

Единый протокол оценки

- Split: test_size=0.2, random_state=42, stratify=y.
- Test остаётся закрытым до финального дня.
- Разработка: StratifiedKFold(n_splits=5, shuffle=True, random_state=42) только на train.
- Основные метрики: F1 и ROC-AUC; дополнительно accuracy, precision и recall.
- Baseline: DummyClassifier(strategy='most_frequent').
- Одинаковые folds и scoring для всех сравнений.

Ежедневный Git-ритуал

```
git status  
# после Restart & Run All:  
git add week-4 README.md LEARNING_LOG.md  
git commit -m "week4 day4: add leakage-safe ML pipeline"  
git push
```

- Один commit - один логический результат.
- Перед push проверь notebook, ссылки, графики и git diff --check.
- Не смешивай реорганизацию прошлых недель с PR недели 4.

День 1 - Аудит данных и протокол оценки

Цель дня

Понять, какие колонки требуют разных преобразований, и заранее зафиксировать честный способ сравнения моделей.

Что понять

- Числовой dtype не всегда означает числовой смысл: Pclass можно считать категорией.
- Идентификатор не является полезным признаком сам по себе.
- Пропуски надо изучить до выбора imputation-стратегии.
- Holdout test нужен для финальной проверки, а не для ежедневного выбора решения.

Материалы

- scikit-learn: Getting Started - transformers, preprocessing и Pipeline.
- scikit-learn: Common pitfalls - inconsistent preprocessing и data leakage.
- Свои ноутбуки Week 3 day 3, day 4 и day 6 - baseline, метрики и CV.

Задания (сделать руками)

- Создай структуру week-4 и notebook day01_preprocessing_audit.ipynb.
- Загрузи Titanic, выведи shape, head, info, долю пропусков, уникальные значения и баланс target.
- Собери таблицу: колонка -> роль -> тип -> пропуски -> действие. Отдельно отметь target и идентификаторы.
- Выбери base features без feature engineering: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked.
- Сделай один stratified train/test split и запиши размеры и баланс классов.
- На train оцени baseline через 5-fold StratifiedKFold; test не трогай.

Git-результат дня

- Commit: week4 day1: audit Titanic and freeze evaluation protocol.
- В LEARNING_LOG есть ссылка, список признаков и baseline CV.

День 2 - Пропуски и числовые признаки

Цель дня

Научиться заполнять пропуски и масштабировать числа так, чтобы статистики вычислялись только по train.

Что понять

- `SimpleImputer(strategy='median')` хранит медианы, найденные во время `fit`.
- `StandardScaler` хранит `train mean` и `scale`; `test` только преобразуется.
- Масштабирование важно для `LogisticRegression`, но не обязательно для деревьев.
- Ручной `fit_transform` допустим для демонстрации; в итоговой модели преобразования должны быть внутри `Pipeline`.

Задания (сделать руками)

- Создай `day02_numeric_preprocessing.ipynb` и возьми числовые колонки `Age`, `SibSp`, `Parch`, `Fare`.
- Покажи пропуски и распределения до обработки.
- Обучи `median imputer` на `X_train`; сравни число пропусков до/после.
- Обучи `scaler` на `imputed train`; проверь среднее и стандартное отклонение `train` после `transform`.
- примени только `transform` к `test` и объясни, почему его среднее не обязано быть равно 0.
- Собери `numeric Pipeline`: `imputer -> scaler`. Добавь две проверки через `assert`.

Git-результат дня

- Commit: `week4 day2: add numeric imputation and scaling`.
- В выводах есть разница между `fit_transform` и `transform`.

День 3 - Категориальные признаки

Цель дня

Подготовить текстовые категории для модели и безопасно обработать значение, которого не было в `train`.

Что понять

- `OneHotEncoder` создаёт бинарные колонки для категорий.
- `handle_unknown='ignore'` не даёт `predict` упасть на новой категории.
- Категориальные пропуски можно заполнить самым частым значением или отдельной меткой.
- Число выходных колонок меняется, поэтому полезно смотреть `get_feature_names_out()`.

Задания (сделать руками)

- Создай `day03_categorical_encoding.ipynb`; возьми `Pclass`, `Sex`, `Embarked`.
- Собери `categorical Pipeline`: `most_frequent imputer -> OneHotEncoder`.
- Выведи форму до/после, имена новых признаков и первые преобразованные строки.
- Создай маленький искусственный пример с неизвестной категорией и покажи, что `transform` работает.

Git-результат дня

- Commit: `week4 day3: encode categorical features safely`.

День 4 - ColumnTransformer и единый Pipeline

Цель дня

Объединить числовую и категориальную обработку с LogisticRegression в один объект, который можно безопасно валидировать и применять.

Что понять

- ColumnTransformer направляет разные колонки в разные pipelines.
- Внешний Pipeline связывает preprocessing и estimator.
- При CV каждый fold заново обучает imputer, scaler, encoder и модель только на своей train-части.
- Параметры вложенных шагов называются через двойное подчёркивание: `model__C`.

Материалы

- scikit-learn: Column Transformer with Mixed Types.
- scikit-learn: Pipeline и ColumnTransformer API.
- scikit-learn: Common pitfalls - pipeline как защита от leakage.

Задания (сделать руками)

- Создай `day04_columntransformer_pipeline.ipynb`.
- Определи `numeric_features` и `categorical_features` явно по именам.
- Собери numeric Pipeline: median imputer -> StandardScaler.
- Собери categorical Pipeline: most_frequent imputer -> OneHotEncoder(handle_unknown='ignore').
- Объедини их в ColumnTransformer, затем добавь LogisticRegression(max_iter=1000, random_state=42).
- Оцени baseline и полный pipeline на одном CV: accuracy, precision, recall, F1 и ROC-AUC.
- Выведи mean и std метрик. Test по-прежнему не используй.
- Проверь `get_feature_names_out()` и объясни, сколько признаков стало после preprocessing.

Git-результат дня

- Commit: week4 day4: build leakage-safe preprocessing pipeline.
- В README добавлена таблица baseline vs pipeline по train CV.

Проверка дня

- В коде нет `fit` на полном датасете до CV.
- Один вызов `pipeline.fit` достаточно подготовит данные и обучит модель.
- Pipeline принимает исходный DataFrame, а не вручную созданную матрицу.

День 5 - Feature engineering без утечки

Цель дня

Создать несколько осмысленных признаков Titanic, вынести логику в функцию и честно проверить, помогает ли она модели.

Что понять

- Feature engineering использует знание предметной области, но не должен подсматривать target.
- Сравнивать base и engineered наборы нужно на одинаковых CV-folds.
- Новый признак полезен только если улучшение устойчиво, а логика работает на новых строках.
- Чистая функция получает DataFrame, возвращает копию и не меняет входные данные.

Признаки недели

Признак	Формула/источник	Смысл и проверка
FamilySize	SibSp + Parch + 1	размер семейной группы; минимум 1
IsAlone	FamilySize == 1	бинарный категориальный признак
Title	титул из Name	редкие титулы объединить в Rare
FarePerPerson	Fare / FamilySize	нет деления на ноль; пропуски остаются pipeline

Задания (сделать руками)

- Создай day05_feature_engineering.ipynb и week-4/src/features.py.
- Реализуй функцию add_titanic_features(df); не передавай в неё Survived.
- Добавь проверки: форма не потеряла строки, FamilySize >= 1, входной DataFrame не изменён, нет бесконечности.
- Покажи распределения новых признаков и один содержательный график связи с target на train.
- Собери два одинаковых по модели pipeline: base features и engineered features.
- Сравни их на тех же folds по F1 и ROC-AUC; выведи разницу mean и разброс.
- Если улучшения нет, честно оставь результат и объясни, почему признак всё равно может быть разумным или почему его стоит убрать.

Git-результат дня

- Commit: week4 day5: add tested Titanic features.
- На GitHub есть функция, notebook, график и таблица base vs engineered CV.

День 6 - Cross-validation, несколько метрик и tuning

Цель дня

Подобрать параметры pipeline только на train и понять разницу между метрикой выбора и остальными метриками контроля.

Что понять

- `cross_validate` считает несколько метрик и время обучения.
- `GridSearchCV` перебирает параметры внутри CV; test в подборе не участвует.
- `refit` задаёт метрику, по которой выбирается лучшая комбинация и переобучается финальный estimator на всём train.
- Mean без std неполон: разброс между folds показывает устойчивость.
- Большой разрыв train и validation score может указывать на overfitting.

Задания (сделать руками)

- Создай `day06_cv_tuning.ipynb` и используй лучший pipeline дня 5.
- Задай `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`.
- Через `cross_validate(..., return_train_score=True)` получи accuracy, precision, recall, F1 и ROC-AUC, а также train scores.
- Построй bar chart mean CV с error bars std и сохрани в `week-4/figures/06_cv_metrics.png`.
- Запусти небольшой `GridSearchCV`: `model__C = [0.01, 0.1, 1, 10]`, `model__class_weight = [None, 'balanced']`.
- Используй multi-metric scoring и `refit='roc_auc'`. Сохрани top-5 комбинаций и лучший набор параметров.
- Не смотри test score. Запиши, какую модель перенесёшь в финальный день и почему.

Git-результат дня

- Commit: `week4 day6: tune pipeline with stratified CV`.
- В README есть лучший CV ROC-AUC, F1, std и выбранные параметры.

Красные флаги

- Параметр выбран по лучшему test score.
- Encoder или scaler обучен до `GridSearchCV` на всех данных.
- Для разных моделей использовались разные случайные folds.
- Показана только accuracy без baseline и метрик положительного класса.

День 7 - Итоговый мини-проект и оформление GitHub

Цель дня

Собрать воспроизводимую историю: данные -> split -> pipeline -> CV -> tuning -> один финальный test -> выводы.

Что понять

- Финальный test подтверждает выбранный процесс, но не используется для ещё одного раунда настройки.
- Разница CV и test ожидаема; её надо объяснить, а не скрывать.
- Confusion matrix показывает типы ошибок, ROC-AUC - качество ранжирования, F1 - баланс precision и recall.
- Готовый проект содержит не только число, но и код, протокол оценки, ограничения и способ запуска.

Задания (сделать руками)

- Создай `day07_final_project.ipynb`; убери повторяющийся код в функции и понятные секции.
- Повтори фиксированный train/test split и собери baseline, базовый pipeline и выбранный tuned pipeline.
- Для выбора используй только сохранённую логику дней 5-6. Затем один раз оцени модели на test.
- Посчитай accuracy, precision, recall, F1, ROC-AUC и confusion matrix. Добавь bootstrap не требуется.
- Сохрани `07_confusion_matrix.png` и `07_roc_curve.png`.
- Сделай `reports/week04_summary.md`: задача, данные, split, preprocessing, признаки, CV, параметры, test, ограничения, следующий шаг.
- Обнови README и LEARNING_LOG. Проверь все ссылки и чистоту `git status`.
- Создай pull request week-4 -> main с описанием, таблицей метрик и чек-листом проверки.

Git-результат дня

- Commit: week4 day7: finish preprocessing pipeline project.
- Final push: notebook, функции, графики, отчёт и README согласованы.
- Definition of Done: новый пользователь клонирует репозиторий, ставит зависимости и воспроизводит финальный notebook без ручных правок.

Минимальная итоговая таблица

Модель	Данные	CV F1	CV ROC-AUC	Test F1	Test ROC-AUC
Baseline	без preprocessing	...	...	...	...
Base pipeline	7 исходных признаков	...	...	...	...
Tuned pipeline	engineered features	...	...	...	...

Чек-лист перед сдачей недели

- Есть отдельная ветка/PR Week 4 без случайной переработки старых недель.
- Есть минимум 7 осмысленных коммитов и понятное описание PR.
- Все 7 notebooks валидны и выполняются сверху вниз без ошибок.
- Train/test split выполняется до любого обучаемого preprocessing.
- Imputer, scaler и encoder находятся внутри Pipeline/ColumnTransformer.
- Test не использовался для выбора признаков, модели или параметров.
- Baseline, base pipeline и tuned pipeline сравниваются на одинаковых данных.
- CV использует stratification, shuffle и фиксированный random_state.
- В отчёте есть mean и std CV, финальные test-метрики и ограничения.
- Нет секретов, больших временных файлов, .venv, cache и notebook checkpoints.
- Все ссылки README открываются, графики читаемы, пути работают из корня репозитория.

Шаблон README для Week 4

```
# ML с нуля - Неделя 4

## Цель
Собрать leakage-safe preprocessing pipeline для Titanic.

## Данные и протокол
Target: Survived; test_size=0.2; random_state=42;
test используется один раз в финале.

## Pipeline
numeric: median imputer -> StandardScaler
categorical: most_frequent imputer -> OneHotEncoder
model: LogisticRegression; tuning: train CV only

## Итоги
| Model | CV F1 | CV ROC-AUC | Test F1 | Test ROC-AUC |
|---|---|---|---|---|
| Baseline | ... | ... | ... | ... |
| Base pipeline | ... | ... | ... | ... |
| Tuned pipeline | ... | ... | ... | ... |

## Запуск
pip install -r requirements.txt
jupyter notebook
```

Как писать commit messages

- Плохо: update, try model, final final.
- Хорошо: week4 day2: fit numeric preprocessing on train only.
- Хорошо: week4 day6: compare tuned pipeline on fixed CV folds.

Список источников

- [1] scikit-learn: Getting Started
https://scikit-learn.org/stable/getting_started.html
- [2] scikit-learn: Common pitfalls and recommended practices
https://scikit-learn.org/stable/common_pitfalls.html
- [3] scikit-learn: Column Transformer with Mixed Types
https://scikit-learn.org/stable/auto_examples/compose/plot_column_transformer_mixed_types.html
- [4] scikit-learn: SimpleImputer
<https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>
- [5] scikit-learn: StandardScaler
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [6] scikit-learn: OneHotEncoder
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- [7] scikit-learn: ColumnTransformer
<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>
- [8] scikit-learn: Pipeline
<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>
- [9] scikit-learn: cross_validate
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_validate.html
- [10] scikit-learn: StratifiedKFold
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
- [11] scikit-learn: GridSearchCV
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- [12] scikit-learn: Classification metrics
https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

Куда двигаться после Недели 4

- Сравнить LogisticRegression с DecisionTree и RandomForest внутри того же preprocessing pipeline.
- Разобрать дисбаланс классов: class_weight, порог решения, precision-recall curve.
- Добавить permutation importance и объяснить влияние признаков без причинных выводов.
- Сохранять обученный pipeline и проверять predict на новых строках.
- Начать Неделю 5: ансамбли, интерпретация модели и более строгий экспериментальный отчёт.